



软件源代码缺陷检测

taos-connector-rust

检测报告

苏州棱镜七彩信息科技有限公司

2026-04-17 21:37:48



2004

2010

2012

2006

CHART SEGMENT TITLE

CHART SEGMENT TITLE

报告声明

棱镜七彩是苏州棱镜七彩信息科技有限公司的简称，PrismSAST 是棱镜七彩具有自主知识产权的软件源代码安全缺陷检测平台的简称。

本检测报告由棱镜七彩软件源代码安全缺陷检测平台（PrismSAST）采用智能检测引擎进行自动化检测，并对检测结果经过人工复核后产生的检测报告。棱镜七彩保留对检测报告中各款项的解读权利，但并不意味着对检测结果中的缺陷或安全漏洞可能引起的任何直接经济损失或任何间接损失承担责任。

本检测报告的最终解释权归属棱镜七彩。

1 项目信息

项目名称	taos-connector-rust
提交时间	2026-04-20 17:23:35
完成时间	2026-04-20 17:23:46
用时	00:00:12
代码总行数	2179
检测文件数	14
缺陷总数	41
致命缺陷数	0
缺陷密度（缺陷/KLOC）	18.82
检测类别	常规检测
报告创建日期时间	2026-04-17 21:37:48
检测团队	安全部
版本号	1.3.0

参考标准

*CMMI（软件能力成熟度模型集成）中缺陷密度参考值：		
CMM5 级	< 0.32	国际一流软件研发企业
CMM4 级	< 0.92	
CMM3 级	< 2.39	国内一流软件研发企业
CMM2 级	< 5.52	国内其他软件研发企业
CMM1 级	< 11.95	

2 风险等级

缺陷/安全漏洞严重等级	等级描述
致命	会造成程序崩溃或系统性故障的缺陷；或可能被容易利用且造成系统重大风险的安全漏洞
严重	会影响程序部分功能和性能，或者造成未定义行为的缺陷；或可能被利用且造成较为严重的风险的安全漏洞
一般	会造成对程序功能或性能的轻度影响；或存在可能被利用的低危安全漏洞。
强制	强制性标准是要求必须严格执行的标准条款
建议	建议性标准是企业可参考，自愿采用的标准条款
提示	不会直接影响程序的功能和性能，但这不是良好的代码风格，可能会有潜在的影响。
此等级划分是按照相关行业标准、国家标准，并参考《GB/T30279-2020 信息安全技术网络安全漏洞分类分级指南》进行定义	

3 评分评级

1. 用户可以在项目级别自定义下列三个影响因子的权重及情况：

序号	影响因子	描述
A	是否外部可达	可以分为物理隔绝、内部局域网连接、连接外网三个级别
B	影响程度	可以分为对别的组件或是部门影响较大、有一般的影响、不影响其它组件或是部门
C	缺陷/安全漏洞严重等级	致命、严重、一般、强制、建议、提示

2. 评分计算公式：A*A 权重 + B*B 权重 + C*C 权重

A、B、C 取值在【0，9】，三个权重相加等于 1，因此结果取值范围也在【0，9】；

最后根据公式的结果分级：低危【0，4），中危【4，6），高危【7，9）。

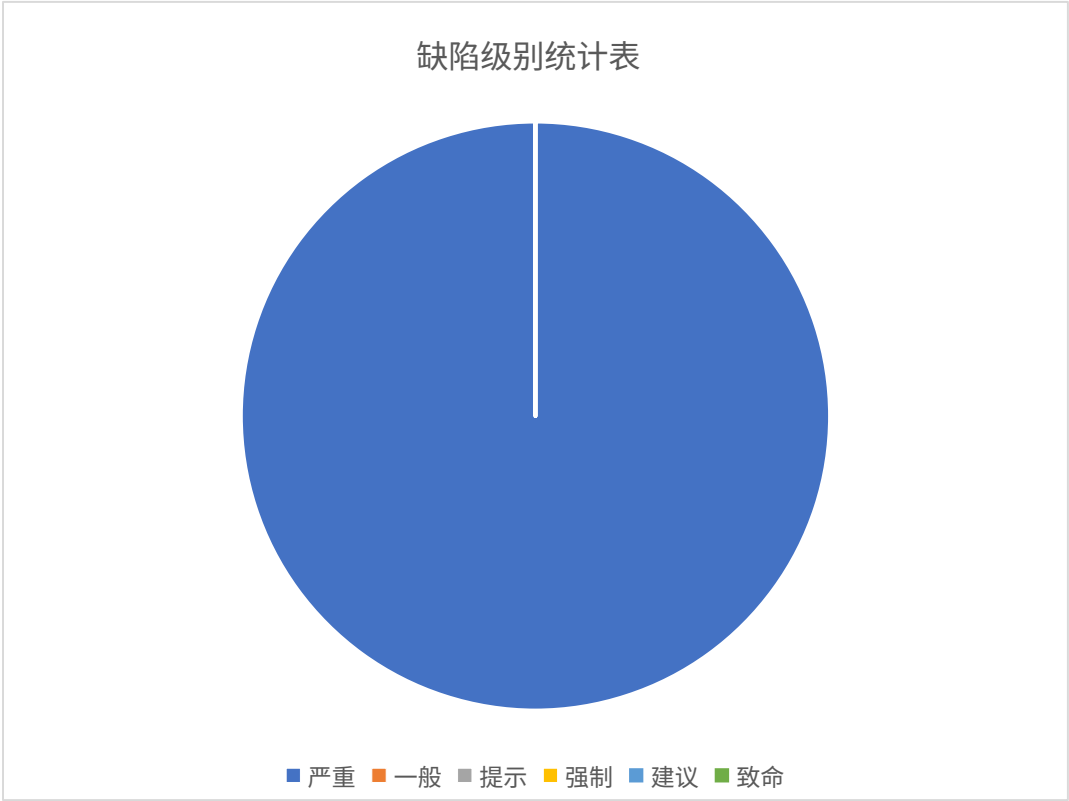
3. 缺省情况下：A 权重 20%，B 权重 20%，C 权重 60%

A = 5，B=5，C：致命=9，严重=7，一般=3，提示=1。

4 汇总统计

4.1 图表统计

4.1.1 按缺陷级别统计



4.1.2 按 CWE 2023 (Top25)统计



4.1.3 按 OWASP 2021 (TOP10)统计



4.2 选择检测集及检测结果统计表

序号	规则集	检测集	选择数量	发现缺陷/安全漏洞数
1	C/C++	CWE C/C++	24	0
2	C/C++缺陷类型	数据加密	3	0
3	C/C++缺陷类型	初始化问题	14	0
4	C/C++缺陷类型	缓冲区溢出	5	0
5	C/C++缺陷类型	空指针解引用	9	0
6	C/C++缺陷类型	并发	2	0
7	C/C++缺陷类型	除零	2	0
8	C/C++缺陷类型	数组问题	6	9
9	C/C++缺陷类型	不匹配的类型转换	11	0
10	C/C++缺陷类型	比较的问题	11	0
11	C/C++缺陷类型	整数溢出	2	0
12	C/C++缺陷类型	字符串问题	6	1
13	C/C++缺陷类型	注入问题	3	0
14	C/C++缺陷类型	返回值问题	10	31
15	C/C++缺陷类型	不可达代码	2	0
16	C/C++缺陷类型	代码质量-头文件问题	7	0
17	C/C++缺陷类型	代码质量-无用数据	8	0
18	C/C++缺陷类型	算术运算	12	0
19	C/C++缺陷类型	恒真恒假	2	0
20	C/C++缺陷类型	跨站攻击	1	0
21	C/C++缺陷类型	浮点数问题	2	0
22	C/C++缺陷类型	不安全密码	4	0
23	C/C++缺陷类型	日志管理	2	0
24	C/C++缺陷类型	资源泄露	3	0
25	C/C++缺陷类型	使用已释放的内存	4	0
26	C/C++缺陷类型	其他内存相关问题	10	0

4.3 人工复核结果统计表

缺陷级别	真实缺陷	误报缺陷	待确认缺陷	忽略缺陷	总计
致命	0	0	0	0	0
严重	0	0	41	0	41
一般	0	0	0	0	0
提示	0	0	0	0	0

强制	0	0	0	0	0
建议	0	0	0	0	0

4.4 代码质量度量统计表

变量描述	数量	简述
总行数	2179	源代码行数统计
总文件数	14	文件数量统计
总类个数	0	编译后中间文件个数
总方法个数	33	所有的方法的个数
总有效代码行数	1808	有效代码行数>=总代码行数-注释行数-空行数 总行数 "c= a+b; //some comments" 如上面一行代码会同时归入有效代码行和注释行
总注释行数	137	总注释行=文件头注释+方法头注释+方法内注释
文件头注释总行数	14	文件头注释
函数声明注释总函数	72	方法函数注释，域变量注释
方法内注释总行数	51	方法内部的注释
总空行数	242	空行数
注释比率	6.28	注释比率=注释行总数/代码总行数
声明注释比率	3.98	声明注释比率=头注释行数/有效代码
代码注释比例	2.82	代码注释比例=函数内注释行数/有效代码
总基本块数	0	基本块数
出度为 1 的基本块数	0	出度为 1 的基本块数
总操作数	3	加，减，乘，除，取余，与，或，异或等操作
参与操作符运算的变量数	3	参与操作符运算的变量数
程序出口个数	14	程序出口个数
扇入数比率	0.00	扇入数/函数数量
扇出数比率	0.00	扇入数/函数数量
扇入数	0	当前节点的所有被调用数量
扇出数	0	当前节点的所有调用数量
循环数	25	程序中所有的 loop 操作数量
圈复杂度	6.84	$(E-N+2)/\text{总方法数量}$

5 C/C++缺陷类型-数组问题

5.1 分项汇总表

序号	C/C++缺陷类型-数组问题	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4672	数组定义禁止没有显式的边界限定	9	严重

5.2 缺陷/安全漏洞详解

5.2.1 严重类

5.2.1.1 WK4672 数组定义禁止没有显式的边界限定

缺陷数量：9 个		
缺陷描述： 数组定义禁止没有显式的边界限定。		
修复意见： 无		
1	缺陷发生位置： /taos-ws-sys/examples/ws-muti-bind.c	行号： 102
	步骤描述： 数组 current 定义的没有显式的边界限定	
	代码片段： 100: 1648432615749, 101: 1648432616249}; 102: float current[] = { 103: 10.3, 104: 12.6,	

	污染路径： 无	
2	缺陷发生位置： /taos-ws-sys/examples/ws-table-muti-bind.c	行号： 106
	步骤描述： 数组 phase 定义的没有显式的边界限定	
	代码片段： 104: 220, 105: 222}; 106: float phase[] = { 107: 0.31, 108: 0.33,	
	污染路径： 无	
3	缺陷发生位置： /taos-ws-sys/examples/ws-table-muti-bind.c	行号： 95
	步骤描述： 数组 voltage 定义的没有显式的边界限定	
	代码片段： 93: 12.0, 94: 10.1}; 95: int voltage[] = { 96: 219, 97: 218,	
	污染路径： 无	
4	缺陷发生位置： /taos-ws-sys/examples/ws-muti-bind.c	行号： 136
	步骤描述： 数组 loc2 定义的没有显式的边界限定	
	代码片段： 134: 0.30}; 135: 136: char loc2[] = {'a', 'b', 0, 'c', 'd', 0, 'e', 'f', 0, 'g', 'h', 0, 'i', 'j', 0, 'a', 'b', 0, 'c', 'd', 0, 'e', 'f', 0, 'g', 'h', 0, 'i', 'j', 0}; 137: char loc3[1000] = {0}; 138: int nchar_len = strlen("一二三四五六七八");	
	污染路径： 无	

5	缺陷发生位置： /taos-ws-sys/examples/ws-table-muti-bind.c	行号： 73
	步骤描述： 数组 ts 定义的没有显式的边界限定	
	代码片段： 71: // values to bind 72: int8_t data_count = 10; 73: int64_t ts[] = { 74: 1648432611249, 75: 1648432611749,	
	污染路径： 无	
6	缺陷发生位置： /taos-ws-sys/examples/ws-muti-bind.c	行号： 113
	步骤描述： 数组 voltage 定义的没有显式的边界限定	
	代码片段： 111: 12.0, 112: 10.1}; 113: int voltage[] = { 114: 219, 115: 218,	
	污染路径： 无	
7	缺陷发生位置： /taos-ws-sys/examples/ws-muti-bind.c	行号： 124
	步骤描述： 数组 phase 定义的没有显式的边界限定	
	代码片段： 122: 220, 123: 222}; 124: float phase[] = { 125: 0.31, 126: 0.33,	
	污染路径： 无	
8	缺陷发生位置： /taos-ws-sys/examples/ws-muti-bind.c	行号： 91
	步骤描述： 数组 ts 定义的没有显式的边界限定	

	代码片段: 89: // values to bind 90: int8_t data_count = 10; 91: int64_t ts[] = { 92: 1648432611249, 93: 1648432611749, 	
	污染路径: 无	
9	缺陷发生位置: /taos-ws-sys/examples/ws-table-muti-bind.c	行号: 84
	步骤描述: 数组 current 定义的没有显式的边界限定	
	代码片段: 82: 1648432615749, 83: 1648432616249}; 84: float current[] = { 85: 10.3, 86: 12.6, 	
	污染路径: 无	

6 C/C++缺陷类型-字符串问题

6.1 分项汇总表

序号	C/C++缺陷类型-字符串问题	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4778	用于表示字符串的数组必须以\0 结束	1	严重

6.2 缺陷/安全漏洞详解

6.2.1 严重类

6.2.1.1 WK4778 用于表示字符串的数组必须以 \0 结束

缺陷数量：1 个		
缺陷描述： 用于表示字符串的数组必须以 \0 结束		
修复意见： 无		
1	缺陷发生位置： /taos-ws-sys/examples/ws-muti-bind.c	行号： 136
	步骤描述： 用于表示字符串的数组必须以 '\0' 结束.	
	代码片段： 134: 0.30}; 135: 136: char loc2[] = {'a', 'b', 0, 'c', 'd', 0, 'e', 'f', 0, 'g', 'h', 0, 'i', 'j', 0, 'a', 'b', 0, 'c', 'd', 0, 'e', 'f', 0, 'g', 'h', 0, 'i', 'j', 0}; 137: char loc3[1000] = {0}; 138: int nchar_len = strlen("一二三四五六七八");	
	污染路径： 无	

7 C/C++缺陷类型-返回值问题

7.1 分项汇总表

序号	C/C++缺陷类型-返回值问题	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4733	具有返回值的函数,其返回值如果不被使用,调用是应有 void 说明	31	严重

7.2 缺陷/安全漏洞详解

7.2.1 严重类

7.2.1.1 WK4733 具有返回值的函数,其返回值如果不被使用,调

用是应有 void 说明

缺陷数量： 31 个		
缺陷描述： 具有返回值的函数,其返回值如果不被使用,调用时应有 void 说明。		
修复意见： 无		
1	缺陷发生位置： /taos-ws-sys/examples/show-databases.c	行号： 14
	步骤描述： 不使用返回值的函数'ws_enable_log'应使用'void'说明.	
	代码片段：	

	12: dsn = "ws://localhost:6041"; 13: } 14: ws_enable_log("debug"); 15: WS_TAOS *taos = ws_connect(dsn); 16: if (taos == NULL)	
	污染路径: 函数'ws_enable_log'的定义点.	
2	缺陷发生位置: /taos-ws-sys/examples/ws-interface.c	行号: 39
	步骤描述: 不使用返回值的函数'dprintf'应使用'void'说明.	
	代码片段: 37: { 38: const char *errstr = ws_errstr(taos); 39: dprintf(2, "Error [%6x]: %s \n", code, errstr); 40: ws_free_result(rs); 41: ws_close(taos);	
	污染路径: 无	
3	缺陷发生位置: /taos-ws-sys/examples/ws-fetch-row.c	行号: 155
	步骤描述: 不使用返回值的函数'dprintf'应使用'void'说明.	
	代码片段: 153: { 154: const char *errstr = ws_errstr(rs); 155: dprintf(2, "Error [%6x]: %s \n", code, errstr); 156: ws_free_result(rs); 157: ws_close(taos);	
	污染路径: 无	
4	缺陷发生位置: /taos-ws-sys/examples/ws-tmqr-all.c	行号: 326
	步骤描述: 不使用返回值的函数'ws_enable_log'应使用'void'说明.	
	代码片段: 324: int32_t code; 325: 326: ws_enable_log("debug"); 327:	

	328: if (init_env() < 0)	
	污染路径: 函数'ws_enable_log'的定义点.	
5	缺陷发生位置: /taos-ws-sys/examples/ws-interface.c	行号: 21
	步骤描述: 不使用返回值的函数'dprintf'应使用'void'说明.	
	代码片段: 19: int code = ws_errno(NULL); 20: const char *errstr = ws_errstr(NULL); 21: dprintf(2, "Error [%6x]: %s", code, errstr); 22: return 0; 23: }	
	污染路径: 无	
6	缺陷发生位置: /taos-ws-sys/examples/ws-fetch-row.c	行号: 33
	步骤描述: 不使用返回值的函数'dprintf'应使用'void'说明.	
	代码片段: 31: { 32: const char *errstr = ws_errstr(taos); 33: dprintf(2, "Error [%6x]: %s \n", code, errstr); 34: ws_free_result(rs); 35: ws_close(taos);	
	污染路径: 无	
7	缺陷发生位置: /taos-ws-sys/examples/ws-err-code.c	行号: 17
	步骤描述: 不使用返回值的函数'ws_enable_log'应使用'void'说明.	
	代码片段: 15: 16: int errno = 0; 17: ws_enable_log("debug"); 18: WS_TAOS *taos = ws_connect(dsn); 19:	
	污染路径: 函数'ws_enable_log'的定义点.	

8	缺陷发生位置： /taos-ws-sys/examples/ws-get-err.c	行号： 168
	步骤描述： 不使用返回值的函数'ws_enable_log'应使用'void'说明.	
	代码片段： 166: int main() 167: { 168: ws_enable_log("debug"); 169: WS_TAOS *taos = ws_connect("ws://localhost:6041"); 170: if (taos == NULL)	
	污染路径： 函数'ws_enable_log'的定义点.	
9	缺陷发生位置： /taos-ws-sys/examples/ws-schemaless.c	行号： 40
	步骤描述： 不使用返回值的函数'dprintf'应使用'void'说明.	
	代码片段： 38: const char *errstr = ws_errstr(rs); 39: printf("-----%ld-----1\n", errstr); 40: dprintf(2, "Error [%6x]: %s", code, errstr); 41: return code; 42: }	
	污染路径： 无	
10	缺陷发生位置： /taos-ws-sys/examples/ws-cloud.c	行号： 10
	步骤描述： 不使用返回值的函数'ws_enable_log'应使用'void'说明.	
	代码片段： 8: int main() 9: { 10: ws_enable_log("debug"); 11: 12: char *url = getenv("TDENGINE_CLOUD_URL");	
	污染路径： 函数'ws_enable_log'的定义点.	
11	缺陷发生位置： /taos-ws-sys/examples/show-databases.c	行号： 25
	步骤描述： 不使用返回值的函数'dprintf'应使用'void'说明.	

	代码片段: 23: 24: const char *version = ws_get_server_info(taos); 25: dprintf(2, "Server version: %s\n", version); 26: 27: WS_RES *rs = ws_query(taos, "show databases");	
	污染路径: 无	
12	缺陷发生位置: /taos-ws-sys/examples/ws-basic.c	行号: 19
	步骤描述: 不使用返回值的函数'dprintf'应使用'void'说明.	
	代码片段: 17: 18: const char *version = ws_get_server_info(taos); 19: dprintf(2, "Server version: %s\n", version); 20: 21: WS_RES *res1 = ws_query(taos, "drop database basic");	
	污染路径: 无	
13	缺陷发生位置: /taos-ws-sys/examples/show-databases.c	行号: 29
	步骤描述: 不使用返回值的函数'dprintf'应使用'void'说明.	
	代码片段: 27: WS_RES *rs = ws_query(taos, "show databases"); 28: int64_t timing = ws_take_timing(rs); 29: dprintf(2, "Query timing: %lldns\n", timing); 30: int code = ws_errno(rs); 31: if (code != 0)	
	污染路径: 无	
14	缺陷发生位置: /taos-ws-sys/examples/ws-interface.c	行号: 28
	步骤描述: 不使用返回值的函数'dprintf'应使用'void'说明.	
	代码片段: 26: if (r != 0) 27: { 28: dprintf(2, "Error: use db return: %d\n", r);	

	29: return 0; 30: }	
	污染路径: 无	
15	缺陷发生位置: /taos-ws-sys/examples/ws-fetch-row.c	行号: 14
	步骤描述: 不使用返回值的函数'ws_enable_log'应使用'void'说明.	
	代码片段: 12: dsn = "ws://localhost:6041"; 13: } 14: ws_enable_log("debug"); 15: WS_TAOS *taos = ws_connect(dsn); 16: if (taos == NULL)	
	污染路径: 函数'ws_enable_log'的定义点.	
16	缺陷发生位置: /taos-ws-sys/examples/show-databases.c	行号: 20
	步骤描述: 不使用返回值的函数'dprintf'应使用'void'说明.	
	代码片段: 18: int code = ws_errno(NULL); 19: const char *errstr = ws_errstr(NULL); 20: dprintf(2, "Error [%6x]: %s", code, errstr); 21: return 0; 22: }	
	污染路径: 无	
17	缺陷发生位置: /taos-ws-sys/examples/ws-fetch-row.c	行号: 20
	步骤描述: 不使用返回值的函数'dprintf'应使用'void'说明.	
	代码片段: 18: int code = ws_errno(NULL); 19: const char *errstr = ws_errstr(NULL); 20: dprintf(2, "Error [%6x]: %s", code, errstr); 21: return 0; 22: }	
	污染路径:	

	无	
18	缺陷发生位置: /taos-ws-sys/examples/ws-bind.c	行号: 147
	步骤描述: 不使用返回值的函数'ws_enable_log'应使用'void'说明.	
	代码片段: 145: int main() 146: { 147: ws_enable_log("debug"); 148: WS_TAOS *taos = ws_connect("ws://localhost:6041"); 149: if (taos == NULL)	
	污染路径: 函数'ws_enable_log'的定义点.	
19	缺陷发生位置: /taos-ws-sys/examples/show-databases.c	行号: 34
	步骤描述: 不使用返回值的函数'dprintf'应使用'void'说明.	
	代码片段: 32: { 33: const char *errstr = ws_errstr(taos); 34: dprintf(2, "Error [%6x]: %s", code, errstr); 35: ws_free_result(rs); 36: ws_close(taos);	
	污染路径: 无	
20	缺陷发生位置: /taos-ws-sys/examples/ws-schemaless.c	行号: 15
	步骤描述: 不使用返回值的函数'ws_enable_log'应使用'void'说明.	
	代码片段: 13: dsn = "ws://localhost:6041/"; 14: } 15: ws_enable_log("trace"); 16: WS_TAOS *taos = ws_connect(dsn); 17: assert(taos != NULL);	
	污染路径: 函数'ws_enable_log'的定义点.	
21	缺陷发生位置: /taos-ws-sys/examples/ws-cloud.c	行号: 19
	步骤描述:	

	不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 17: 18: char dsn[1024]; 19: snprintf(dsn, sizeof(dsn), "%s/rust_test?token=%s", url, token); 20: 21: WS_TAOS *taos = ws_connect(dsn);	
	污染路径: 无	
22	缺陷发生位置: /taos-ws-sys/examples/ws-basic.c	行号: 15
	步骤描述: 不使用返回值的函数'ws_enable_log'应使用'void'说明.	
	代码片段: 13: dsn = "ws://localhost:6041"; 14: } 15: ws_enable_log("debug"); 16: WS_TAOS *taos = ws_connect(dsn); 17:	
	污染路径: 函数'ws_enable_log'的定义点.	
23	缺陷发生位置: /taos-ws-sys/examples/ws-interface.c	行号: 15
	步骤描述: 不使用返回值的函数'ws_enable_log'应使用'void'说明.	
	代码片段: 13: dsn = "ws://localhost:6041"; 14: } 15: ws_enable_log("trace"); 16: WS_TAOS *taos = ws_connect(dsn); 17: if (taos == NULL)	
	污染路径: 函数'ws_enable_log'的定义点.	
24	缺陷发生位置: /taos-ws-sys/examples/ws-interface.c	行号: 34
	步骤描述: 不使用返回值的函数'dprintf'应使用'void'说明.	
	代码片段: 32: WS_RES *rs = ws_query(taos, "select * from d0 limit 1");	

	33: int64_t timing = ws_take_timing(rs); 34: dprintf(2, "Query timing: %ldns\n", timing); 35: int code = ws_errno(rs); 36: if (code != 0)	
	污染路径: 无	
25	缺陷发生位置: /taos-ws-sys/examples/ws-schemaless.c	行号: 59
	步骤描述: 不使用返回值的函数'dprintf'应使用'void'说明.	
	代码片段: 57: { 58: const char *errstr = ws_errstr(taos); 59: dprintf(2, "Error [%6x]: %s", code, errstr); 60: return code; 61: }	
	污染路径: 无	
26	缺陷发生位置: /taos-ws-sys/examples/ws-run-sql.c	行号: 35
	步骤描述: 不使用返回值的函数'dprintf'应使用'void'说明.	
	代码片段: 33: { 34: const char *errstr = ws_errstr(taos); 35: dprintf(2, "Error [%6x]: %s", code, errstr); 36: ws_free_result(rs); 37: ws_close(taos);	
	污染路径: 无	
27	缺陷发生位置: /taos-ws-sys/examples/ws-run-sql.c	行号: 20
	步骤描述: 不使用返回值的函数'ws_enable_log'应使用'void'说明.	
	代码片段: 18: dsn = "ws://localhost:6041"; 19: } 20: ws_enable_log("debug"); 21: WS_TAOS *taos = ws_connect(dsn); 22: if (taos == NULL)	

	污染路径: 函数'ws_enable_log'的定义点.	
28	缺陷发生位置: /taos-ws-sys/examples/ws-basic.c	行号: 32
	步骤描述: 不使用返回值的函数'dprintf'应使用'void'说明.	
	代码片段: <pre> 30: { 31: const char *errstr = ws_errstr(taos); 32: dprintf(2, "Error [%6x]: %s", code, errstr); 33: ws_free_result(res3); 34: ws_close(taos); </pre>	
	污染路径: 无	
29	缺陷发生位置: /taos-ws-sys/examples/ws-run-sql.c	行号: 26
	步骤描述: 不使用返回值的函数'dprintf'应使用'void'说明.	
	代码片段: <pre> 24: int code = ws_errno(NULL); 25: const char *errstr = ws_errstr(NULL); 26: dprintf(2, "Error [%6x]: %s", code, errstr); 27: return 0; 28: } </pre>	
	污染路径: 无	
30	缺陷发生位置: /taos-ws-sys/examples/ws-basic.c	行号: 27
	步骤描述: 不使用返回值的函数'dprintf'应使用'void'说明.	
	代码片段: <pre> 25: WS_RES *res3 = ws_query(taos, "create table basic.t (ts timestamp, v int, name varchar(20), mark nchar(20))"); 26: int64_t timing = ws_take_timing(res3); 27: dprintf(2, "Query timing: %lldns\n", timing); 28: int code = ws_errno(res3); 29: if (code != 0) </pre>	
	污染路径: 无	

31	缺陷发生位置： /taos-ws-sys/examples/ws-tmq-basic.c	行号： 15
	步骤描述： 不使用返回值的函数'ws_enable_log'应使用'void'说明.	
	代码片段： 13: dsn = "ws://localhost:6041"; 14: } 15: ws_enable_log("debug"); 16: 17: ws_tmq_conf_t *conf = ws_tmq_conf_new();	
	污染路径： 函数'ws_enable_log'的定义点.	